

UNIVERSIDAD AUTONOMA DE AGUASCALIENTES

**INGENIERIA EN COMPUTACION
INTELIGENTE**

**ADMINISTRACION DE SOFTWARE Y
PROYECTOS**



PLANIFICACION SPRINT 5

Arrioja Pizaña Cesar Emmanuel

Cruz Mora Ángel

Semestre VIII

Segmentación Operativa y Gestión de Ciclo de Vida Doble

El sprint de cierre se enfocó en las reglas de negocio e inteligencia especializada de la compañía. En primer lugar, se programó el ***Sistema de Filtrado por División***, permitiendo separar rigurosamente las órdenes correspondientes a estructuras pesadas (*Events*) del mobiliario o mantelería fina (*Party*) para una distribución idónea de los camiones. En segundo lugar, se codificó el control de la doble etapa del ciclo logístico: la Entrega (*Delivery*) y la Recolección (*Pick-up*). Esto garantiza que el sistema no dé por terminada una orden hasta que los activos retornen con éxito al almacén, maximizando el control del inventario real y cerrando el flujo del MVP 100% libre de papel.

Para esto mismo, el objetivo del sprint es consolidar el núcleo del backend operativo mediante la gestión avanzada de contratos/órdenes en PostgreSQL (usando Drizzle ORM), implementar el aprovisionamiento de personal administrativo y de conducción con sincronización en Clerk, e interconectar el ciclo de asignaciones con la mensajería asíncrona de Apache Kafka para alimentar la reactividad en tiempo real de la plataforma.

LOGROS ALCANZADOS (A GRANDES RASGOS): Se programó el módulo de lógica y filtrado exclusivo para segmentar la operación entre las divisiones de *Events* y *Party*.

Hito A: Motor Transaccional de Órdenes y Manifiestos de Carga

- **Resolución Dinámica de Ítems (Polimorfismo de Inventario):** Implementación de una capa inteligente capaz de discriminar e integrar dinámicamente ítems del manifiesto provenientes tanto de partyProducts como de eventProducts (filtrando atributos específicos como *size* y *style*) bajo una única estructura unificada de respuesta para el cliente.
- **Seguridad Transaccional en Asignaciones:** Orquestación de transacciones SQL (db.transaction) para garantizar la atomicidad en la asignación de transportistas. La operación asegura en un único paso atómico: la mutación de estado a ASSIGNED, el purgado de asignaciones huérfanas previas, el enlace del nuevo conductor y el registro de auditoría local (eventLogs).

Hito B: Aprovisionamiento y Balanceo de Carga de Personal (Admin & Drivers)

- **Módulo CRUD de Empleados:** Creación de un sistema de altas, bajas y modificaciones de personal operativo en base de datos local con soporte de escalabilidad de roles dinámicos.
- **Cálculo de Workload en Tiempo Real:** Integración de agregaciones SQL nativas insertadas en las consultas de Drizzle (sql<number>) para mapear la carga de trabajo exacta de cada operador (activeTasksCount), omitiendo las tareas

completadas de forma reactiva para facilitar el balanceo de asignaciones desde la oficina.

- **Pre-Aprovisionamiento Híbrido (Clerk Gateway Ready):** Diseño de un mecanismo de inserción temprana mediante IDs temporales autogenerados (temp_...). Esto permite dar de alta a un empleado por correo electrónico antes de que este inicie sesión; el sistema enlazará el ID real de Clerk en el primer *handshake*.

Hito C: Puentes de Integración Kafka (Event-Driven Pipeline)

- **Estrategia Productora de Ciclo de Vida:** Vinculación directa del flujo transaccional con Apache Kafka. Cada asignación exitosa de una orden de entrega despacha de inmediato un evento estructurado al topic `tasks.lifecycle.assigned`.

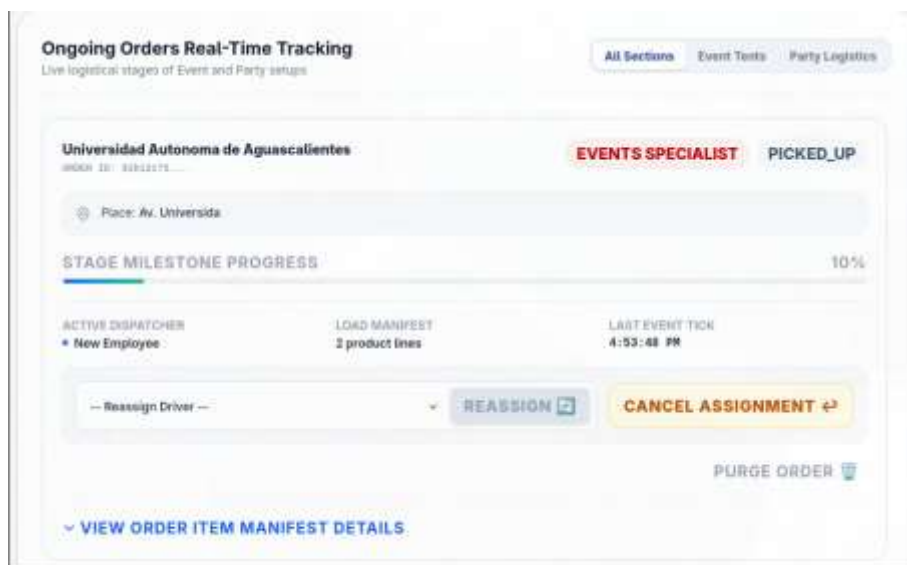
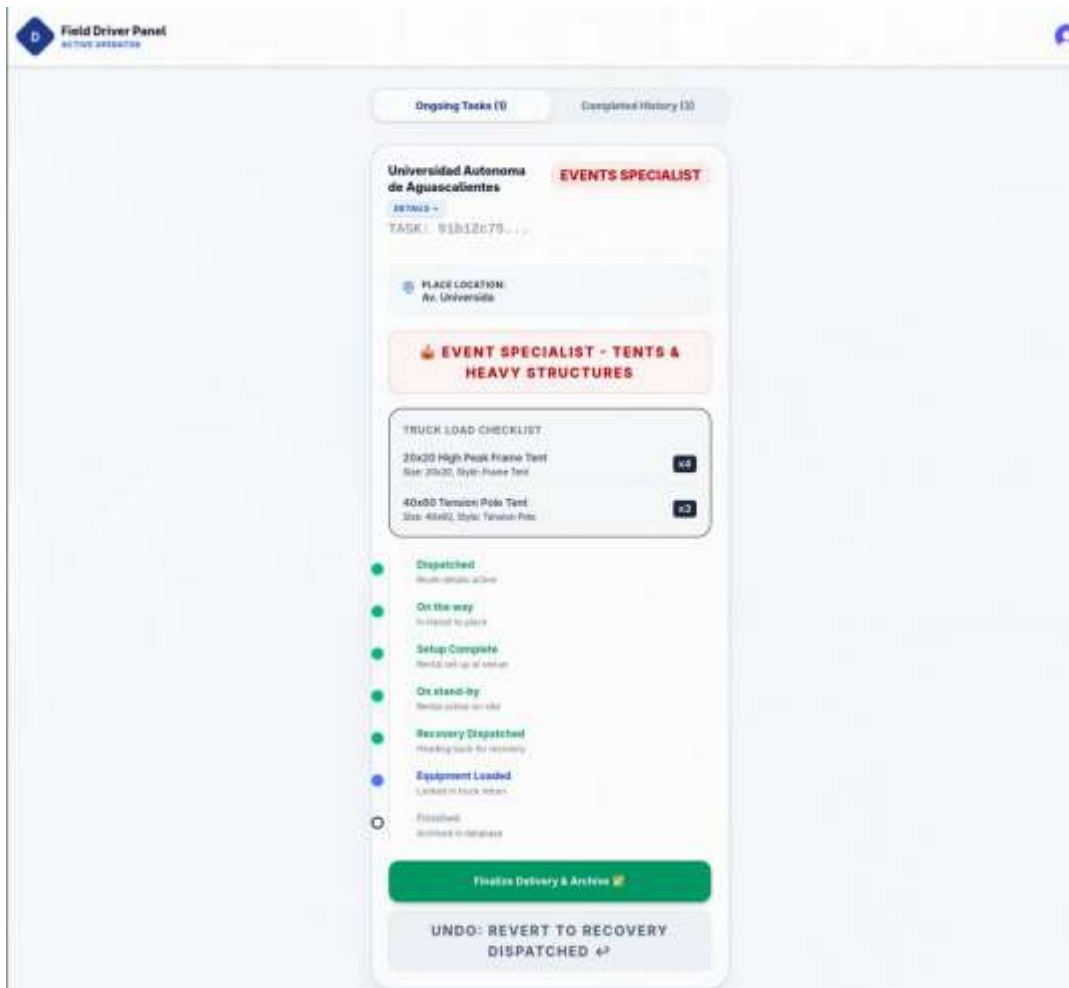
A esto, tenemos aquí un desglose de los métodos y endpoints del sistema(que hacen)

Método	Endpoint	Rol Requerido	Descripción Operativa
GET	/api/orders?orderId=...	OFFICE / DRIVER	Inspecciona una orden detallada resolviendo dinámicamente el manifiesto de inventario.
GET	/api/orders?userId=...&role=OFFICE	OFFICE	Retorna dos colecciones ordenadas: Órdenes Activas y Contratos Pendientes no asignados.
GET	/api/orders?userId=...&role=DRIVER	DRIVER	Retorna estrictamente el manifiesto de tareas asignadas al transportista autenticado.
POST	/api/orders	OFFICE	Ejecuta la transacción

			atómica de asignación de ruta y emite el evento correspondiente a Kafka.
DELETE	/api/orders?orderId=...&action=cancel	OFFICE / ADMIN	Libera una orden asignada, destruyendo los enlaces del conductor y regresándola al pool de pendientes.
GET	/api/users?role=DRIVER	OFFICE	Muestra el catálogo de operadores con el contador inyectado de tareas pendientes.
POST	/api/users	ADMIN	Provisiona cuentas de empleados en el sistema bajo un esquema híbrido de ID temporal.

ENTREGABLES

Evidencia:



Código:

```
import { NextRequest, NextResponse } from "next/server";
import { db } from "@src/lib/db/db";
import {
  orders,
  orderAssignments,
  clients,
  orderItems,
  partyProducts,
  eventProducts,
  users,
  eventLogs
} from "@src/lib/db/schema";
import { produceEvent } from "@src/lib/kafka/client";
import { eq, and, isNull, sql } from "drizzle-orm";

export const dynamic = "force-dynamic";

// Helper to fetch and resolve item details (name, sku, category) for a given order
async function fetchOrderItems(orderId: string) {
  const itemsList = await db
    .select({
      id: orderItems.id,
      productId: orderItems.productId,
      productType: orderItems.productType,
      quantity: orderItems.quantity
    })
    .from(orderItems)
    .where(eq(orderItems.orderId, orderId));

  const resolvedItems = await Promise.all(
    itemsList.map(async (item) => {
      let name = "Unknown Item";
      let sku = "N/A";
      let detail = "";

      if (item.productType === "PARTY") {
        const prod = await db
          .select({ name: partyProducts.name, sku: partyProducts.sku })
          .from(partyProducts)
          .where(eq(partyProducts.id, item.productId))
          .limit(1);
        if (prod.length > 0) {
          name = prod[0].name;
          sku = prod[0].sku;
        }
      } else {
        const prod = await db
          .select({ name: eventProducts.name, sku: eventProducts.sku, size: eventProducts.size, style:
eventProducts.style })
          .from(eventProducts)
          .where(eq(eventProducts.id, item.productId))
          .limit(1);
        if (prod.length > 0) {
          name = prod[0].name;

```

```

        sku = prod[0].sku;
        detail = `Size: ${prod[0].size || ""}, Style: ${prod[0].style || ""}`;
    }
}

return {
    ...item,
    name,
    sku,
    detail
};
})
);

return resolvedItems;
}

// 1. GET: Fetch ongoing/unassigned orders
export async function GET(req: NextRequest) {
    try {
        const { searchParams } = new URL(req.url);
        const orderId = searchParams.get("orderId");

        if (orderId) {
            const orderRecord = await db
                .select({
                    id: orders.id,
                    division: orders.division,
                    status: orders.status,
                    pickupDate: orders.pickupDate,
                    createdAt: orders.createdAt,
                    updatedAt: orders.updatedAt,
                    driverId: orderAssignments.driverId,
                    driverName: users.name,
                    clientName: clients.name,
                    clientPhone: clients.phone,
                    clientEmail: clients.email,
                    clientAddress: clients.address,
                })
                .from(orders)
                .leftJoin(orderAssignments, eq(orders.id, orderAssignments.orderId))
                .leftJoin(users, eq(orderAssignments.driverId, users.id))
                .innerJoin(clients, eq(orders.clientId, clients.id))
                .where(eq(orders.id, orderId))
                .limit(1);

            if (orderRecord.length === 0) {
                return NextResponse.json({ error: "Order not found" }, { status: 404 });
            }

            const items = await fetchOrderItems(orderId);
            return NextResponse.json({
                success: true,
                order: { ...orderRecord[0], items }
            });
        }
    }
}

```

```

const userId = searchParams.get("userId");
const role = searchParams.get("role"); // 'OFFICE' | 'DRIVER'

if (!userId || !role) {
  return NextResponse.json({ error: "Missing userId or role" }, { status: 400 });
}

if (role === "OFFICE") {
  // A. Fetch all active/completed orders (anything that is NOT status 'PENDING')
  const activeOrdersList = await db
    .select({
      id: orders.id,
      division: orders.division,
      status: orders.status,
      pickupDate: orders.pickupDate,
      createdAt: orders.createdAt,
      updatedAt: orders.updatedAt,
      driverId: orderAssignments.driverId,
      driverName: users.name,
      clientName: clients.name,
      clientPhone: clients.phone,
      clientAddress: clients.address,
    })
    .from(orders)
    .leftJoin(orderAssignments, eq(orders.id, orderAssignments.orderId))
    .leftJoin(users, eq(orderAssignments.driverId, users.id))
    .innerJoin(clients, eq(orders.clientId, clients.id))
    .where(sql` ${orders.status} != 'PENDING'`);

  // B. Fetch all unassigned oncoming orders (status = 'PENDING')
  const oncomingOrdersList = await db
    .select({
      id: orders.id,
      division: orders.division,
      status: orders.status,
      pickupDate: orders.pickupDate,
      createdAt: orders.createdAt,
      updatedAt: orders.updatedAt,
      clientName: clients.name,
      clientEmail: clients.email,
      clientPhone: clients.phone,
      clientAddress: clients.address,
    })
    .from(orders)
    .innerJoin(clients, eq(orders.clientId, clients.id))
    .where(eq(orders.status, "PENDING"));

  // C. Resolve manifest items for active orders
  const activeWithItems = await Promise.all(
    activeOrdersList.map(async (order) => {
      const items = await fetchOrderItems(order.id);
      return { ...order, items };
    })
  );

```

```

// D. Resolve manifest items for oncoming orders
const oncomingWithItems = await Promise.all(
  oncomingOrdersList.map(async (order) => {
    const items = await fetchOrderItems(order.id);
    return { ...order, items };
  })
);

return NextResponse.json({
  success: true,
  activeOrders: activeWithItems,
  unassignedContracts: oncomingWithItems // Kept name unassignedContracts for frontend backwards-
compatibility
});

} else if (role === "DRIVER") {
  // Fetch only active orders assigned to this driver
  const driverOrders = await db
    .select({
      id: orders.id,
      division: orders.division,
      status: orders.status,
      pickupDate: orders.pickupDate,
      createdAt: orders.createdAt,
      updatedAt: orders.updatedAt,
      clientName: clients.name,
      clientPhone: clients.phone,
      clientEmail: clients.email,
      clientAddress: clients.address, // place
    })
    .from(orders)
    .innerJoin(orderAssignments, eq(orders.id, orderAssignments.orderId))
    .innerJoin(clients, eq(orders.clientId, clients.id))
    .where(eq(orderAssignments.driverId, userId));

  // Resolve manifest items for driver's tasks
  const driverOrdersWithItems = await Promise.all(
    driverOrders.map(async (order) => {
      const items = await fetchOrderItems(order.id);
      return { ...order, items };
    })
  );

  return NextResponse.json({
    success: true,
    activeOrders: driverOrdersWithItems
  });
}

return NextResponse.json({ error: "Invalid role" }, { status: 400 });

} catch (error: any) {
  console.error("GET orders error:", error);
  return NextResponse.json({ error: error.message || String(error) }, { status: 500 });
}
}

```

```

// 2. POST: Office Member assigns an oncoming PENDING order to a driver
export async function POST(req: NextRequest) {
  try {
    const body = await req.json();
    const { orderId, driverId } = body; // Expects orderId (oncoming UUID) and driverId (Clerk ID)

    if (!orderId || !driverId) {
      return NextResponse.json({ error: "Missing orderId or driverId" }, { status: 400 });
    }

    // A. Verify the oncoming order exists and is currently PENDING
    const existingOrder = await db
      .select({
        id: orders.id,
        status: orders.status,
        division: orders.division,
        clientId: orders.clientId,
        clientName: clients.name,
        clientAddress: clients.address,
      })
      .from(orders)
      .innerJoin(clients, eq(orders.clientId, clients.id))
      .where(eq(orders.id, orderId))
      .limit(1);

    if (existingOrder.length === 0) {
      return NextResponse.json({ error: "Order not found" }, { status: 404 });
    }

    const orderRecord = existingOrder[0];
    if (orderRecord.status === "COMPLETED") {
      return NextResponse.json({ error: "Cannot re-assign a finished/completed order." }, { status: 400 });
    }

    // B. Fetch driver name to verify they exist
    const driverRecord = await db
      .select({ name: users.name })
      .from(users)
      .where(eq(users.id, driverId))
      .limit(1);

    if (driverRecord.length === 0) {
      return NextResponse.json({ error: "Selected driver not found in the database." }, { status: 404 });
    }

    // Fetch items manifest
    const items = await fetchOrderItems(orderId);

    // C. Perform DB updates inside a transaction
    const now = new Date();
    const updatedOrder = await db.transaction(async (tx) => {
      // 1. Update the Order status to 'ASSIGNED' and reset timestamp
      const [orderRow] = await tx
        .update(orders)
        .set({

```

```

        status: "ASSIGNED",
        updatedAt: now
    })
    .where(eq(orders.id, orderId))
    .returning();

// 2. Delete any existing assignment(s) for this order to enable clean re-assignment
await tx.delete(orderAssignments).where(eq(orderAssignments.orderId, orderId));

// 3. Insert new Assignment
await tx.insert(orderAssignments).values({
    orderId,
    driverId,
    assignedAt: now
});

// 3. Log event locally
await tx.insert(eventLogs).values({
    orderId,
    eventType: "office.order.assigned",
    payload: JSON.stringify({
        driverId,
        driverName: driverRecord[0].name,
        clientId: orderRecord.clientId,
        clientName: orderRecord.clientName,
        items
    }),
    timestamp: now
});

return orderRow;
});

console.log(`[Order Assignment] Order ${orderId} assigned to driver ${driverId}.`);

// D. Produce Event to Kafka: tasks.lifecycle.assigned topic
const eventPayload = {
    orderId,
    driverId,
    division: orderRecord.division,
    clientName: orderRecord.clientName,
    clientAddress: orderRecord.clientAddress,
    items,
    timestamp: now.toISOString()
};

await produceEvent(
    "tasks.lifecycle.assigned",
    "office.order.assigned",
    eventPayload
);

console.log(`[Kafka Produced] office.order.assigned event published to Kafka.`);

return NextResponse.json({
    success: true,

```

```

    message: "Order successfully assigned and published to Kafka!",
    order: updatedOrder
  });

} catch (error: any) {
  console.error("POST orders error:", error);
  return NextResponse.json({ error: error.message || String(error) }, { status: 500 });
}
}

// 3. DELETE: Cancel/Reset order to PENDING or purge it entirely from PostgreSQL
export async function DELETE(req: NextRequest) {
  try {
    const { searchParams } = new URL(req.url);
    const orderId = searchParams.get("orderId");
    const action = searchParams.get("action"); // 'cancel' (reset to PENDING) or 'delete' (purge from DB)

    if (!orderId) {
      return NextResponse.json({ error: "Missing orderId" }, { status: 400 });
    }

    if (action === "delete") {
      // Purge permanently
      await db.transaction(async (tx) => {
        await tx.delete(eventLogs).where(eq(eventLogs.orderId, orderId));
        await tx.delete(orderAssignments).where(eq(orderAssignments.orderId, orderId));
        await tx.delete(orderItems).where(eq(orderItems.orderId, orderId));
        await tx.delete(orders).where(eq(orders.id, orderId));
      });

      console.log(`[Admin] Purged order ${orderId} permanently.`);

      return NextResponse.json({
        success: true,
        message: "Order successfully purged from database."
      });
    } else {
      // Cancel assignment - reset status to PENDING and delete assignments
      await db.transaction(async (tx) => {
        await tx.delete(orderAssignments).where(eq(orderAssignments.orderId, orderId));
        await tx.update(orders)
          .set({ status: "PENDING", updatedAt: new Date() })
          .where(eq(orders.id, orderId));
      });

      console.log(`[Admin] Cancelled assignments for order ${orderId}, reset to PENDING.`);

      return NextResponse.json({
        success: true,
        message: "Order assignment cancelled and returned to oncoming pool."
      });
    }
  } catch (error: any) {
    console.error("DELETE orders error:", error);
    return NextResponse.json({ error: error.message || String(error) }, { status: 500 });
  }
}

```

```

    }
  }
import { NextRequest, NextResponse } from "next/server";
import { db } from "@src/lib/db/db";
import { users, orderAssignments, orders } from "@src/lib/db/schema";
import { eq, and, ne, sql } from "drizzle-orm";

export const dynamic = "force-dynamic";

// 1. GET: Fetch all drivers along with their active task counts (workload)
export async function GET(req: NextRequest) {
  try {
    const { searchParams } = new URL(req.url);
    const userId = searchParams.get("userId");
    const roleFilter = searchParams.get("role"); // e.g. 'DRIVER'

    if (userId && !roleFilter) {
      // Fetch specific user profile (e.g. for dynamic role checking in client side)
      const userRecord = await db
        .select()
        .from(users)
        .where(eq(users.id, userId))
        .limit(1);

      return NextResponse.json({
        success: true,
        user: userRecord[0] || null
      });
    }

    let query = db
      .select({
        id: users.id,
        name: users.name,
        email: users.email,
        role: users.role,
        createdAt: users.createdAt,
        // SQL count of active, incomplete tasks currently assigned
        activeTasksCount: sql<number>`COALESCE(count(CASE WHEN ${orders.status} !=
'COMPLETED' THEN 1 END), 0)::integer`
      })
      .from(users)
      .leftJoin(orderAssignments, eq(users.id, orderAssignments.driverId))
      .leftJoin(orders, eq(orderAssignments.orderId, orders.id))
      .groupBy(users.id);

    // Apply role filter if specified
    if (roleFilter) {
      // Drizzle where
      // @ts-ignore
      query = query.where(eq(users.role, roleFilter));
    }

    const results = await query;

    return NextResponse.json({

```

```

    success: true,
    users: results
  });

  } catch (error: any) {
    console.error("GET users error:", error);
    return NextResponse.json({ error: error.message || String(error) }, { status: 500 });
  }
}

// 2. POST: Admin provisions a new employee profile in PostgreSQL
export async function POST(req: NextRequest) {
  try {
    const body = await req.json();
    const { name, email, role } = body;

    // Validate request parameters
    if (!name || !email || !role) {
      return NextResponse.json({ error: "Missing name, email, or role" }, { status: 400 });
    }

    const allowedRoles = ["OFFICE", "DRIVER"];
    if (!allowedRoles.includes(role)) {
      return NextResponse.json({ error: "Invalid role. Must be 'OFFICE' or 'DRIVER'" }, { status: 400 });
    }

    const normalizedEmail = email.trim().toLowerCase();

    // A. Check if the user email already exists in DB
    const existingUser = await db
      .select()
      .from(users)
      .where(eq(users.email, normalizedEmail))
      .limit(1);

    if (existingUser.length > 0) {
      // Email already exists, let's update their role to the new requested role!
      const [updatedUser] = await db
        .update(users)
        .set({ role: role })
        .where(eq(users.email, normalizedEmail))
        .returning();

      console.log(`[Provisioning] Existing employee '${name}' upgraded/changed to role '${role}'. email=${normalizedEmail}`);

      return NextResponse.json({
        success: true,
        message: `Successfully updated existing account for ${name} to role ${role}!`,
        user: updatedUser
      });
    }

    // B. Insert user record with temporary ID
    // When the user logs in via Clerk for the first time, our gateway
    // will link their real Clerk User ID automatically.

```

```

const tempId = `temp_${Date.now()}_${Math.random().toString(36).substr(2, 9)}`;

const [newUser] = await db
  .insert(users)
  .values({
    id: tempId,
    name: name.trim(),
    email: normalizedEmail,
    role: role,
    createdAt: new Date()
  })
  .returning();

console.log(`[Provisioning] Employee '${name}' (${role}) provisioned in DB.
email=${normalizedEmail}`);

return NextResponse.json({
  success: true,
  message: `Successfully provisioned account for ${name}! They can now log in via Clerk.`,
  user: newUser
});

} catch (error: any) {
  console.error("POST users error:", error);
  return NextResponse.json({ error: error.message || String(error) }, { status: 500 });
}
}

// 3. PUT: Update an employee's data (role, name, email)
export async function PUT(req: NextRequest) {
  try {
    const body = await req.json();
    const { userId, role, name, email } = body;

    if (!userId) {
      return NextResponse.json({ error: "Missing userId" }, { status: 400 });
    }

    const updateFields: any = {};
    if (role) {
      const allowedRoles = ["OFFICE", "DRIVER"];
      if (!allowedRoles.includes(role)) {
        return NextResponse.json({ error: "Invalid role. Must be 'OFFICE' or 'DRIVER'" }, { status: 400 });
      }
      updateFields.role = role;
    }
    if (name !== undefined) {
      if (!name.trim()) {
        return NextResponse.json({ error: "Name cannot be empty" }, { status: 400 });
      }
      updateFields.name = name.trim();
    }
    if (email !== undefined) {
      if (!email.trim() || !email.includes("@")) {
        return NextResponse.json({ error: "Invalid email address" }, { status: 400 });
      }
    }
  }
}

```

```

    updateFields.email = email.trim().toLowerCase();
  }

  if (Object.keys(updateFields).length === 0) {
    return NextResponse.json({ error: "No fields to update" }, { status: 400 });
  }

  // Update role/name/email in Postgres
  const [updatedUser] = await db
    .update(users)
    .set(updateFields)
    .where(eq(users.id, userId))
    .returning();

  if (!updatedUser) {
    return NextResponse.json({ error: "User not found" }, { status: 404 });
  }

  console.log(`[Admin] Updated user ${userId} fields:`, updateFields);

  return NextResponse.json({
    success: true,
    message: `Successfully updated user parameters`,
    user: updatedUser
  });

} catch (error: any) {
  console.error("PUT users error:", error);
  return NextResponse.json({ error: error.message || String(error) }, { status: 500 });
}
}

// 4. DELETE: Remove an employee profile from Postgres
export async function DELETE(req: NextRequest) {
  try {
    const { searchParams } = new URL(req.url);
    const userId = searchParams.get("userId");

    if (!userId) {
      return NextResponse.json({ error: "Missing userId" }, { status: 400 });
    }

    // A. Delete order assignments for this driver first to prevent foreign key errors
    await db.delete(orderAssignments).where(eq(orderAssignments.driverId, userId));

    // B. Delete the user
    const [deletedUser] = await db
      .delete(users)
      .where(eq(users.id, userId))
      .returning();

    if (!deletedUser) {
      return NextResponse.json({ error: "User not found" }, { status: 404 });
    }

    console.log(`[Admin] Deleted user account ${userId}`);
  }
}

```

```
return NextResponse.json({
  success: true,
  message: `Successfully removed user account for ${deletedUser.name}`
});

} catch (error: any) {
  console.error("DELETE users error:", error);
  return NextResponse.json({ error: error.message || String(error) }, { status: 500 });
}
}
```